PONTO 1 · VISÃO HOLÍSTICA

SDLC seguro quando a IA escreve o código

Os princípios da Janca, as técnicas da literatura
e os mecanismos do playbook de SDLC com IA, no mesmo ciclo.

Fontes principais: Janca (2020), *Alice and Bob Learn Application Security* · Claxton (2026), *The AI-native SDLC playbook*. Páginas da Janca: paginação do índice remissivo da edição Kindle.

O MOLDE

Três camadas, da regra à execução

Princípio	Que regra não quebra?	Janca, 2020
Técnica	Qual método nomeado acha o problema?	Literatura (quem, ano)
Mecanismo	O que executa a regra?	Claxton, 2026

A IA entra em cada fase. O humano continua responsável pelas decisões de julgamento.

Camadas: README deste repositório, "Hierarquia de estudo" · Frase: Claxton, "What is an AI-native SDLC?"

A FIGURA DO PONTO 1

Cada fase, suas técnicas

PONTA · HUMANO DEFINE | MEIO · GUARDRAIL AUTOMÁTICO | PONTA · HUMANO APROVA

Requisitos

TÉCNICAS

- Abuse case — McDermott e Fox, 1999
- Misuse case — Sindre e Opdahl, 2000
- Attack tree — Schneier, 1999
- Checklist de requisitos — Janca p. 61–63

MECANISMO IA

`intent.md` revisado pelo product owner
01 Plan

Design

TÉCNICAS

- Modelagem de ameaças — Shostack, 2014 · Janca p. 78–82
- DFD — DeMarco, 1979
- Superfície de ataque — Manadhata e Wing, 2004

MECANISMO IA

`spec.md` sob skill de segurança
02 Design

Código

TÉCNICAS

- Inspeção formal — Fagan, 1976
- Code review — Janca p. 122–123
- SAST — Janca p. 123–125

MECANISMO IA

Modo plano + hooks
03 Build

Testes

TÉCNICAS

- Fuzzing — Miller et al., 1990 · Janca p. 133
- DAST · SCA — Janca p. 133–134, 125–126
- Pentest — Janca p. 135–136

MECANISMO IA

Teste que falha primeiro; evals no CI
04 Test

Revisão e deploy

TÉCNICAS

- Secure code review — OWASP · Janca p. 122–123
- Separação de funções — Claxton, 05 Deploy

MECANISMO IA

Review contra `REVIEW.md`; code owner aprova
05 Deploy

Observabilidade

TÉCNICAS

- Resposta a incidentes — NIST SP 800-61 · Janca p. 200–201
- Fire drill — Janca p. 201–202
- Varredura contínua — Janca p. 202

MECANISMO IA

1σ · 2σ · 3σ → `intent.md`
06 Maintain

O agente está em todas as fases. **A atenção humana se concentra nas pontas.**

Pontas: Claxton, "Code is no longer the bottleneck" (o gargalo vai para antes e depois do build) e "Plays" (a atenção humana se concentra nos portões) · Mecanismos: Claxton, seção indicada em cada coluna.

ONDE FICA O HUMANO

O agente está em todas as fases. A decisão humana fica nas pontas.

INÍCIO

Agente: escreve `intent.md` e `spec.md`, sinaliza conflito de política

Humano: o product owner revisa; o dono da política decide

MEIO

Agente: implementa a partir do `plan.md` aprovado

Sistema: hooks e evals seguram

FIM

Agente: revisa o PR, diagnostica o alerta

Humano: code owner aprova; uma pessoa nomeada libera produção

Claxton: "01 Plan" (product owner revisa o intent) · "02 Design" (conflito vai ao dono) · "03 Build" (plan.md, hooks) · "04 Test" (evals) · "05 Deploy" (code owner, portão de produção) · "06 Maintain" (2σ diagnostica).

O LIVRO, NUM QUADRO

Onze capítulos, duas partes

PARTE	CAPÍTULOS
I · O que saber para escrever código seguro o bastante para a internet	1 Fundamentos · 2 Requisitos · 3 Design · 4 Código · 5 Armadilhas comuns
II · O que fazer para criar código muito bom	6 Testes e deploy · 7 Programa de AppSec · 8 Sistemas modernos · 9 Bons hábitos · 10 Aprendizado contínuo · 11 Considerações finais

Fases do SDLC no livro: Requisitos, Design, Código, Testes e Release.

Janca: sumário (títulos das partes e capítulos) · fases do SDLC, p. 21.

PUSHING LEFT

Falha descoberta tarde custa mais



Bug nasce no código

Falha de design nasce no desenho

Janca: Introdução, "Pushing Left" · falha × bug, p. 66–68 · custo e descoberta tardia da falha, p. 67 · pushing left e falhas de design, p. 68. Barras ilustrativas, sem valores do livro.

O QUE A IA MUDA

Código deixou de ser o gargalo

O gargalo vai para **antes e depois do build**

Controles feitos para código humano **deixam de bater**

Exceção esperando comitê **encarece a governança**

Falha de design descoberta tarde **continua cara**

Claxton, "Code is no longer the bottleneck" (três primeiros) · Janca p. 67 (quarto).

CAPÍTULO 1 · SECURITY FUNDAMENTALS

Doze fundamentos

CIA
p. 3–6

Assume breach
p. 7–8

Ameaça interna
p. 8–9

Defense in depth
p. 9–10

Least privilege
p. 11

Cadeia de suprimentos
p. 11–13

Sem obscuridade
p. 13–14

Menos superfície de ataque
p. 14–15

Sem hard coding
p. 15

Never trust, always verify
p. 15–17

Segurança usável
p. 17–18

Fatores de autenticação
p. 18–19

Janca, cap. 1: seções do sumário, páginas do índice remissivo.

O CICLO

Cada fase deixa um artefato

Requisitos	Design	Código	Testes	Release
PLAN	DESIGN	BUILD	TEST	DEPLOY + MAINTAIN
`intent.md`	`spec.md`	`plan.md + diff`	`teste + eval`	`PR · incidente`

↺ o diagnóstico em produção volta como `intent.md`

Fases: Janca p. 21 · Estágios e artefatos: Claxton, "The shifts across the six stages" · Volta do loop: Claxton, "06 Maintain · Closing the loop".

RASTREABILIDADE

Cada estágio commita o que o próximo lê

`intent.md` → `spec.md` → `plan.md` → `diff + testes` → `PR revisado` → `incidente`

A cadeia de commits é a **trilha de auditoria**.

Claxton, "What is an AI-native SDLC?".

FASE 1 · REQUISITOS ↔ PLAN PONTA

O que o sistema deve garantir

NA JANCA

Requisito de segurança no projeto
p. 22–23

Checklist de requisitos
p. 61–63

Security user stories
p. 178

TÉCNICAS

Abuse case
McDermott e Fox, 1999

Misuse case
Sindre e Opdahl, 2000

Attack tree
Schneier, 1999

NO PLAYBOOK

Quem tem a ideia conversa com o agente e gera o `intent.md`

Product owner revisa antes do commit

Janca: páginas indicadas · Técnicas: autores indicados · Playbook: Claxton, "01 Plan · Capture as intent.md".

FASE 2 · DESIGN PONTA

Desenho seguro antes do código

NA JANCA

Least privilege
p. 11, 60–61

Zero trust e assume breach no design
p. 70–71

Dado de produção segregado
p. 77–78

TÉCNICAS

Modelagem de ameaças
Shostack, 2014 · Janca p. 78–82

STRIDE · PASTA
Janca p. 79

DFD
DeMarco, 1979

NO PLAYBOOK

`spec.md` gerada a partir do intent, guiada por skill de segurança

Conflito com política vai para o dono

Janca: páginas indicadas · Playbook: Claxton, "02 Design · Requirements and design".

FASE 3 · CÓDIGO ↔ BUILD

Sem repetir falha conhecida

NA JANCA

Dado não confiável
p. 87–89

Validação de entrada
p. 58–59

Identidade e sessão
p. 90–92

Bounds checking
p. 93–94

TÉCNICAS

Inspeção formal
Fagan, 1976

Code review
Janca p. 122–123

SAST
Janca p. 123–125

NO PLAYBOOK

Modo plano lê sem editar; plano aprovado vira `plan.md`

Hook bloqueia caminho protegido e credencial no diff

Janca: páginas indicadas · Playbook: Claxton, "03 Build · plan mode" e "Hooks as build-time guardrails".

CAPÍTULO 4 NA PRÁTICA

Dois casos do livro

UPLOAD DE ARQUIVO · P. 56–57

Assume o pior

Valida tipo e tamanho; renomeia

O usuário não escolhe o caminho

Guarda longe, escaneia, aceita só tipos menos perigosos

ERRO E LOG · P. 57

Nada de stack trace na tela

Fail-closed: desfaz a transação e fecha o que abriu

Loga para quem responde e para quem audita

No formato do SIEM

No playbook, o sandbox também falha fechado: não sobe se não conseguir isolar.

Janca p. 56–57 · Claxton, "05 Deploy · Worked example: managed settings".

FASE 4 · TESTES ↔ TEST

Testar o sistema, não só o código

NA JANCA

Testar o código
p. 122–128

Testar a aplicação rodando
p. 129–136

Infra, banco, APIs, integrações, rede
p. 141–145

TÉCNICAS

Fuzzing
Miller et al., 1990 · Janca p. 133

DAST · SCA
Janca p. 133–134, 125–126

Pentest
Janca p. 135–136

NO PLAYBOOK

Teste que falha primeiro, protegido contra edição

Todo incidente vira eval permanente no CI

Janca: páginas indicadas · Playbook: Claxton, "04 Test · Give Claude a feedback loop" e "Continuous evals in CI".

CAPÍTULO 6 · O QUE SE TESTA

Sete alvos, não só o código

Código
p. 122–128

Aplicação
p. 129–136

Infraestrutura
p. 141

Banco de dados
p. 141–142

APIs e web services
p. 142–143

Integrações
p. 143–144

Rede
p. 144–145

+ Deploy
p. 145–146

No playbook, o agente verifica o próprio trabalho com teste, build ou comparação de tela.

Janca, cap. 6: páginas indicadas · Claxton, "04 Test · Give Claude a feedback loop".

FASE 5 · RELEASE ↔ DEPLOY + MAINTAIN PONTA

Detectar e responder em produção

NA JANCA

Assume breach
p. 7–8

Log sem dado sensível
p. 57

Monitoramento
p. 101–103

TÉCNICAS

Resposta a incidentes
NIST SP 800-61 · Janca p. 200–201

Varredura contínua
Janca p. 202

NO PLAYBOOK

Agente vai até o portão de produção; uma pessoa nomeada libera

1σ registra · 2σ diagnostica · 3σ propõe PR

Janca: páginas indicadas · Playbook: Claxton, "05 Deploy · CI/CD integration and deployment" e "06 Maintain · Closing the loop".

CAPÍTULO 9 · BONS HÁBITOS

O hábito no livro e o mecanismo no playbook

JANCA	CLAXTON
Fire drill · p. 201–202	Rollback é o caminho mais ensaiado · 05 Deploy
Varredura contínua · p. 202	Scans agendados em todos os repositórios · 06 Maintain
Resposta a incidentes · p. 200–201	Agente no canal do incidente; post-mortem vai para um arquivo de lições versionado · 06 Maintain

Janca e Claxton nas páginas e seções de cada linha. O pareamento das linhas é deste material.

PRINCÍPIOS DA JANCA × MANAGED SETTINGS

Os mesmos princípios, aplicados ao agente

PRINCÍPIO (JANCA)	GUARDRAIL (CLAXTON, 05 DEPLOY)
Least privilege · p. 11	Token de CI curto e escopado, sem credencial de produção
Sem hard coding · p. 15	Leitura de `.env` e credenciais negada
Fail-closed · p. 57	Sandbox não sobe se não conseguir isolar
Ameaça interna · p. 8–9	O agente que escreveu não aprova; sem rota para a main
Cadeia de suprimentos · p. 11–13	Só hooks, marketplaces e MCP gerenciados

Janca: páginas indicadas · Claxton, "05 Deploy": "AI in the PR review loop", "CI/CD integration and deployment", "Worked example: managed settings". O pareamento das linhas é deste material.

CONTROLE

Aconselhar não é impedir

CONTROLE	MECANISMO	EFEITO
Advisory	Skill	Conformidade **provável**
Enforced	Hook, permissão, branch protection	Violação **quase impossível**

Regra que sempre tem que valer precisa de trava determinística.
Na Janca: **Security as Code**.

Claxton, "03 Build · Skills as institutional knowledge" e "Hooks as build-time guardrails" · Janca, Security as Code, p. 128–129, 177–178.

O QUE PRECISA SER TRABALHADO

Quatro frentes onde a atenção humana pesa

1 · REQUISITOS

Checklist de requisitos no `intent.md`

Janca p. 61–63 · Claxton, 01 Plan

2 · DEFINIÇÃO DE SEGURANÇA

Modelagem de ameaças na `spec.md`; política como skill com dono

Janca p. 78–82 · Claxton, 02 Design

3 · REVISÃO

Critério de segurança no `REVIEW.md`; code owner aprova

Janca p. 122–123 · Claxton, 05 Deploy

4 · OBSERVABILIDADE

Monitoramento, log sem dado sensível, fire drill

Janca p. 57, 101–103, 201–202 · Claxton, 06 Maintain

Por que estas quatro: Claxton, "Code is no longer the bottleneck" e "Plays". A escolha das frentes é deste material.

O loop continua rodando.
O julgamento humano fica acima dele.

Janca, T. (2020). *Alice and Bob Learn Application Security*. Wiley.

Claxton, L. (2026). *The AI-native SDLC playbook*. Anthropic. claude.com/resources/articles/the-ai-native-sdlc-playbook

Shostack, A. (2014). *Threat Modeling*. Wiley.

McDermott, J.; Fox, C. (1999). Abuse case models.

Sindre, G.; Opdahl, A. (2000). Misuse cases.

Schneier, B. (1999). Attack trees.

DeMarco, T. (1979). *Structured Analysis*.

Manadhata, P.; Wing, J. (2004). Attack surface.

Fagan, M. (1976). Design and code inspections.

Miller, B. et al. (1990). Fuzz testing.

NIST SP 800-61. Incident handling guide.

OWASP Code Review Guide.

Frase: Claxton, "Closing thoughts" (tradução).